

# OpenClaw for Agentic AI

A Practitioner's Tutorial Note

*with Hands-On Python Examples*

🦞 *The Lobster Way* 🦞

**Dr. Anish Roychowdhury**

Gen AI Architect | AI Educator | Practitioner

*anish.roychowdhury@gmail.com*

May 2026

**GitHub Repository**

<https://github.com/anishiisc/openclaw>

All code artifacts referenced in this note are available in the repository above.

# Contents

---

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction: The Agent Inflection Point</b> | <b>2</b>  |
| 1.1      | A Brief History . . . . .                       | 2         |
| 1.2      | Why This Tutorial? . . . . .                    | 2         |
| <b>2</b> | <b>Architecture: How OpenClaw Works</b>         | <b>3</b>  |
| 2.1      | The Key Distinction . . . . .                   | 3         |
| 2.2      | Architecture Overview . . . . .                 | 3         |
| 2.3      | The Skills System . . . . .                     | 3         |
| 2.4      | Capabilities at a Glance . . . . .              | 4         |
| <b>3</b> | <b>Installation Guide</b>                       | <b>4</b>  |
| 3.1      | Prerequisites . . . . .                         | 4         |
| 3.2      | Method 1: Install via npm (Canonical) . . . . . | 4         |
| 3.3      | Method 2: Install from Source . . . . .         | 5         |
| 3.4      | Method 3: Python SDK Only . . . . .             | 5         |
| 3.5      | Critical Security Step . . . . .                | 5         |
| <b>4</b> | <b>Code Artifacts: Hands-On Python Examples</b> | <b>5</b>  |
| 4.1      | Artifact 1: Your First OpenClaw Agent . . . . . | 6         |
| 4.2      | Artifact 2: Building a Custom Skill . . . . .   | 7         |
| 4.3      | Artifact 3: Multi-Agent Pipeline . . . . .      | 8         |
| 4.4      | Artifact 4: Agent Loop from Scratch . . . . .   | 9         |
| <b>5</b> | <b>Cost Awareness</b>                           | <b>11</b> |
| <b>6</b> | <b>Security: Eyes Wide Open</b>                 | <b>11</b> |
| 6.1      | Known Risk Categories . . . . .                 | 12        |
| 6.2      | Mitigations . . . . .                           | 12        |
| <b>7</b> | <b>The Bigger Picture: Why OpenClaw Matters</b> | <b>12</b> |
| <b>8</b> | <b>Getting Started Checklist</b>                | <b>13</b> |
| <b>9</b> | <b>References</b>                               | <b>13</b> |

# 1 Introduction: The Agent Inflection Point

There is a moment in every technology wave when a single project crystallises everything that has been building underneath the surface. In late 2025, that moment arrived — wearing the shell of a lobster.

## What is OpenClaw?

**OpenClaw** is a free, open-source autonomous AI agent framework that transforms Large Language Models from conversational assistants into **actionable digital workers**. Unlike ChatGPT or Claude’s chat interface — which answer questions but don’t interact directly with your environment — OpenClaw can read files, execute shell commands, browse websites, send emails, manage calendars, and automate multi-step workflows.

## 1.1 A Brief History

OpenClaw’s journey to becoming GitHub’s fastest-growing project is itself remarkable:

- **November 2025:** Austrian developer Peter Steinberger launches “Clawdbot” as a weekend project.
- **January 27, 2026:** Renamed to “Moltbot” following trademark complaints from Anthropic.
- **January 30, 2026:** Renamed again to “OpenClaw” — the lobster theme persists.
- **February 2026:** Crosses 100,000 GitHub stars. Steinberger joins OpenAI; project transitions to an independent open-source foundation.
- **March 2026:** Surpasses 250,000 stars, overtaking React’s decade-long record in just 60 days.

## Key Insight

**Jensen Huang at GTC 2026:** “OpenClaw is probably the single most important release of software... probably ever.” He compared its significance to Linux, Kubernetes, and HTML, adding: “Claude Code and OpenClaw have sparked the agent inflection point, extending AI beyond generation and reasoning into action. Every company now needs to have an OpenClaw strategy.”

## 1.2 Why This Tutorial?

This tutorial note provides a practitioner’s guide to understanding, installing, and building with OpenClaw. It is designed for:

1. **AI practitioners** who want to understand the agent paradigm shift
2. **Developers** who want to build their first OpenClaw agent
3. **Workshop participants** who need hands-on code examples

All four code artifacts referenced in this document are available at:

<https://github.com/anishiisc/openclaw>

## 2 Architecture: How OpenClaw Works

### 2.1 The Key Distinction

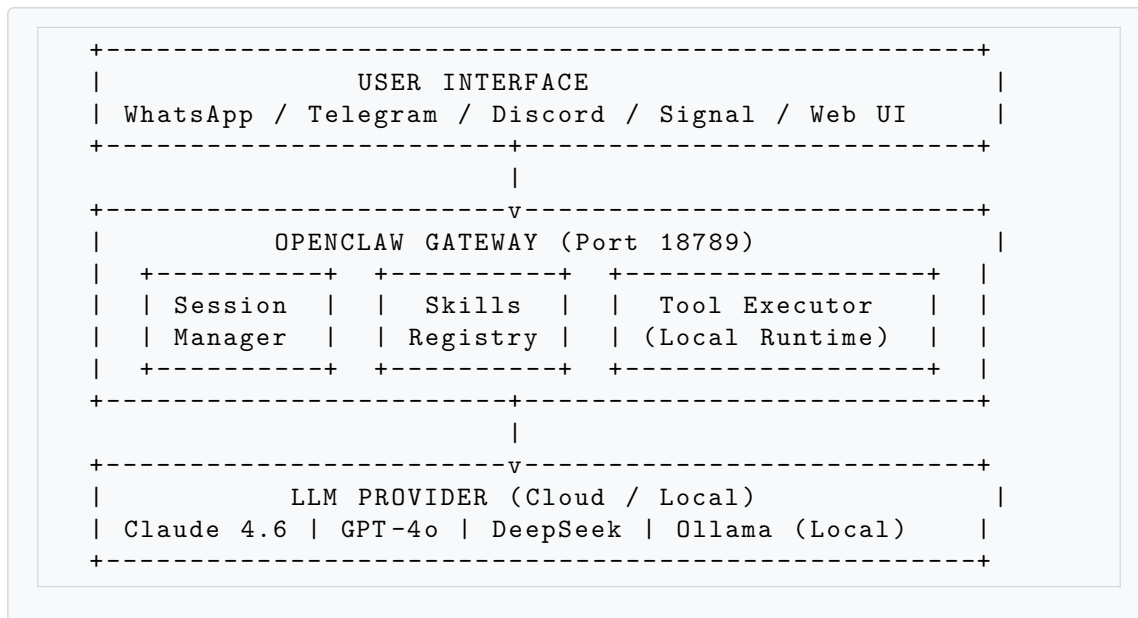
Unlike traditional chatbots, OpenClaw separates the **reasoning layer** (cloud LLM) from the **execution layer** (local runtime). This is architecturally significant:

#### Reasoning vs. Execution

- **Reasoning Layer:** Runs on a cloud LLM — Claude, GPT, DeepSeek, or local models via Ollama. This is where planning, tool selection, and decision-making happen.
- **Execution Layer:** Runs locally on your machine. OpenClaw inherits the same permissions as your user account. Files, shell commands, browser automation, and API calls execute here.

### 2.2 Architecture Overview

The high-level architecture follows a gateway-mediated pattern:



### 2.3 The Skills System

OpenClaw's extensibility comes from its **skills architecture**. Skills are plugin directories containing a `SKILL.md` file with metadata and instructions for the LLM.

#### Skill Directory Structure

```

skills/
  my-custom-skill/
    SKILL.md           # Metadata + LLM instructions
    tool.py            # Tool implementation
    requirements.txt    # Python dependencies
    README.md          # Human-readable docs
  
```

The ecosystem already offers 100+ prebuilt skills covering browser control, GitHub

automation, email management, calendar sync, and smart home integration. Developers publish custom skills to **ClawHub**, the community skills registry.

## 2.4 Capabilities at a Glance

| Use Case                       | Complexity   | Skills Needed         |
|--------------------------------|--------------|-----------------------|
| Summarise emails & auto-reply  | Beginner     | Email skill           |
| Web scraping & data extraction | Beginner     | Browser skill         |
| Manage GitHub PRs & issues     | Intermediate | GitHub skill          |
| Calendar management via chat   | Intermediate | Calendar skill        |
| Multi-step research workflows  | Advanced     | Browser + File skills |
| Autonomous code review         | Advanced     | GitHub + Code skills  |
| Custom skill development       | Expert       | SDK + SKILL.md        |

## 3 Installation Guide

### 3.1 Prerequisites

| Requirement | Details                                  |
|-------------|------------------------------------------|
| Node.js     | v24 recommended (v22.14+ also supported) |
| Python      | 3.9+ (for the Python SDK)                |
| LLM API Key | Anthropic, OpenAI, Google, or others     |
| RAM         | 4 GB minimum, 8 GB+ recommended          |
| OS          | macOS, Linux, Windows (WSL2 recommended) |

### 3.2 Method 1: Install via npm (Canonical)

Terminal: Canonical OpenClaw Installation

```
# Install OpenClaw globally
SHARP_IGNORE_GLOBAL_LIBVIPS=1 npm install -g openclaw@latest

# Run the onboarding wizard (~2 minutes)
openclaw onboard

# Start the gateway
openclaw start

# Open the Control UI
openclaw ui
```

The onboarding wizard walks you through selecting a model provider (Anthropic recommended), entering your API key, and configuring the gateway. Once complete, the gateway listens on port 18789.

### 3.3 Method 2: Install from Source

Terminal: Source Installation

```
# Clone the repository
git clone https://github.com/openclaw/openclaw.git
cd openclaw

# Create a virtual environment
python3 -m venv venv
source venv/bin/activate # Windows: .\venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt

# Configure
cp config/config.example.yaml config/config.yaml
# Edit config.yaml with your API keys

# Run setup
python setup.py configure
```

### 3.4 Method 3: Python SDK Only

For programmatic interaction (the approach used in our code artifacts):

Terminal: SDK-Only Installation

```
pip install openclaw-sdk
```

The SDK auto-detects a running gateway on 127.0.0.1:18789, or you can set the `OPENCLAW_GATEWAY_WS_URL` environment variable explicitly.

### 3.5 Critical Security Step

#### Security: Fix Canvas Host Binding

OpenClaw's Canvas Host component defaults to binding 0.0.0.0, meaning **any device on your local network** can access your OpenClaw interface. Fix this immediately in `~/.openclaw/openclaw.json`:

```
{
  "gateway": {
    "bind": "loopback",
    "port": 18789
  }
}
```

**Additional context:** Gartner analysts called OpenClaw's design "insecure by default." A Kaspersky audit in January 2026 found 512 vulnerabilities, 8 classified as critical. Always run OpenClaw in a sandboxed environment or on a dedicated machine.

## 4 Code Artifacts: Hands-On Python Examples

All code artifacts are available in the companion GitHub repository:

<https://github.com/anishiisc/openclaw>

Each artifact is designed to run on a **laptop** or **Google Colab**. All include dual-mode operation (SDK mode + standalone fallback), comprehensive docstrings, and token/cost tracking.

## 4.1 Artifact 1: Your First OpenClaw Agent

### 01\_openclaw\_basic\_agent.py

**Purpose:** Connects to a local OpenClaw gateway and executes a simple research task. Falls back gracefully to standalone mode using the Anthropic SDK if no gateway is available.

**Key concepts:** Gateway connection, agent creation, task execution, streaming responses, token usage tracking.

#### Artifact 1 — Core Pattern (Excerpt)

```
1 import asyncio
2 from openclaw_sdk import OpenClawClient
3
4 async def run_with_sdk():
5     """Connect to OpenClaw gateway and execute a task."""
6     async with OpenClawClient.connect() as client:
7         # Get a reference to an agent
8         agent = client.get_agent("bot")
9
10        # Execute a research task
11        task = (
12            "List 3 key differences between ReAct and "
13            "Chain-of-Thought prompting for LLM agents."
14        )
15        result = await agent.execute(task)
16        print(result.content)
17
18        # Stream a follow-up question
19        async for event in agent.execute_stream(
20            "Which approach is better for tool-using agents?"
21        ):
22            if hasattr(event, "text"):
23                print(event.text, end="", flush=True)
24
25 asyncio.run(run_with_sdk())
```

### Pro Tip

The standalone fallback (included in the full artifact) uses the Anthropic SDK directly — the same LLM backbone that OpenClaw uses internally. This makes the code runnable in Google Colab without any gateway setup.

## 4.2 Artifact 2: Building a Custom Skill

### 02\_openclaw\_skill\_creator.py

**Purpose:** Programmatically generates a complete “Stock Price Checker” skill with proper SKILL.md metadata, tool implementation, and demonstrates how skills integrate into the OpenClaw ecosystem.

**Key concepts:** SKILL.md schema, tool implementation, skill triggers, ClawHub publishing convention.

The heart of every OpenClaw skill is its SKILL.md file. This tells the LLM *when* and *how* to invoke the skill:

#### SKILL.md — Generated by Artifact 2

```
---
name: stock-price-checker
description: Check real-time stock prices
version: 1.0.0
triggers:
  - stock price
  - check stock
  - how is [TICKER] doing
  - what's the price of [TICKER]
---

# Stock Price Checker Skill

## When to Use
Use this skill when the user asks about:
- Current stock prices for any publicly traded company
- Basic market data (open, close, high, low, volume)

## How to Use
python tool.py --ticker AAPL
python tool.py --tickers AAPL,GOOGL,MSFT --detail
```

#### Artifact 2 — Tool Implementation (Excerpt)

```
1 import yfinance as yf
2 import json
3 from datetime import datetime
4
5 def fetch_stock_price(ticker: str, detail=False):
6     """Fetch stock price data using yfinance."""
7     stock = yf.Ticker(ticker.upper())
8     hist = stock.history(period="1d")
9     latest = hist.iloc[-1]
10
11     result = {
12         "ticker": ticker.upper(),
13         "price": round(float(latest["Close"]), 2),
14         "open": round(float(latest["Open"]), 2),
15         "high": round(float(latest["High"]), 2),
16         "low": round(float(latest["Low"]), 2),
17         "volume": int(latest["Volume"]),
18         "timestamp": datetime.now().isoformat(),
19     }
```



```

20
21     if detail:
22         info = stock.info
23         result.update({
24             "name": info.get("longName", "N/A"),
25             "sector": info.get("sector", "N/A"),
26             "market_cap": info.get("marketCap", "N/A"),
27         })
28     return result

```

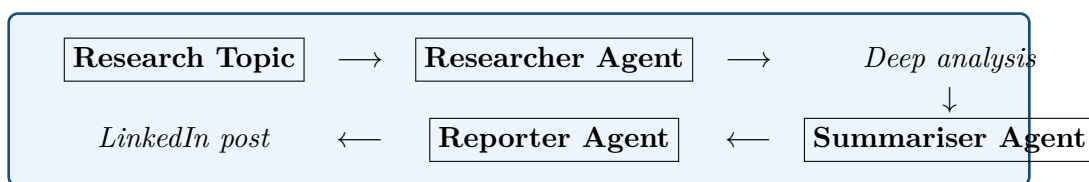
### 4.3 Artifact 3: Multi-Agent Pipeline

#### 03\_openclaw\_multi\_agent\_pipeline.py

**Purpose:** Chains three specialised agents into a composable pipeline: **Researcher** → **Summariser** → **Reporter**. Each agent's output feeds into the next, producing a LinkedIn-ready post from a raw research topic.

**Key concepts:** Agent specialisation, pipeline orchestration, system prompt design, token/cost tracking across stages.

The pipeline architecture mirrors how production OpenClaw deployments orchestrate multi-step workflows:



Artifact 3 — Agent Configuration (Excerpt)

```

1 from dataclasses import dataclass
2
3 @dataclass
4 class AgentConfig:
5     name: str
6     role: str
7     system_prompt: str
8     model: str = "claude-sonnet-4-6"
9     max_tokens: int = 1024
10
11 RESEARCHER = AgentConfig(
12     name="Researcher",
13     role="Deep Research Analyst",
14     system_prompt="""You are a deep research analyst.
15     Structure your research with clear sections.
16     Include specific data points, dates, and names.
17     Output should be 300-500 words."""
18 )
19
20 SUMMARISER = AgentConfig(
21     name="Summariser",
22     role="Executive Summariser",
23     system_prompt="""Extract the 3-5 most important
24     findings. For each: the insight, why it matters,
25     and what to do. Keep under 200 words."""
26 )

```

```

27
28 REPORTER = AgentConfig(
29     name="Reporter",
30     role="LinkedIn Content Formatter",
31     system_prompt="""Start with a compelling hook.
32     Short paragraphs. Include 3-5 hashtags.
33     Professional but approachable. 150-250 words."""",
34 )

```

### Artifact 3 — Pipeline Execution (Excerpt)

```

1 class AgentPipeline:
2     def run(self, topic: str) -> PipelineReport:
3         """Each agent's output feeds into the next."""
4         report = PipelineReport(topic=topic)
5         current_input = topic
6
7         for i, agent in enumerate(self.agents):
8             if i == 0:
9                 prompt = f"Research thoroughly:\n{current_input}"
10            else:
11                prompt = (
12                    f"Previous agent output:\n---\n"
13                    f"{current_input}\n---\n"
14                    f"Apply your expertise."
15                )
16            result = self._call_agent(agent, prompt)
17            report.add_result(result)
18            current_input = result.output_text
19
20        return report

```

## 4.4 Artifact 4: Agent Loop from Scratch

### 04\_openclaw\_agent\_from\_scratch.py

**Purpose:** Implements the core agent loop pattern from first principles — the same **ReAct** (Reasoning + Acting) architecture that powers OpenClaw internally. No OpenClaw SDK required; uses only the Anthropic SDK.

**Key concepts:** Tool-calling loop, Anthropic tool-use API, tool schemas, safety sandboxing, iterative reasoning.

**Inspired by:** The excellent claw0 teaching repository (<https://github.com/shareAI-lab/claw0>).

### Key Insight

This is the **educational centrepiece** of this tutorial. Understanding this loop is understanding how *all* agentic AI systems work — from OpenClaw to Claude Code to AutoGPT. The pattern is: **User Task** → **LLM Reasoning** → **Tool Selection** → **Tool Execution** → **Observation** → **Continue or Finish**.

### Artifact 4 — Tool Definitions (Excerpt)

```

1 TOOLS = [
2     {
3         "name": "calculator",

```

```

4     "description": "Perform mathematical calculations.",
5     "input_schema": {
6         "type": "object",
7         "properties": {
8             "expression": {
9                 "type": "string",
10                "description": "Mathematical expression",
11            }
12        },
13        "required": ["expression"],
14    },
15 },
16 {
17     "name": "file_writer",
18     "description": "Write content to a local file.",
19     "input_schema": {
20         "type": "object",
21         "properties": {
22             "filename": {"type": "string"},
23             "content": {"type": "string"},
24         },
25         "required": ["filename", "content"],
26     },
27 },
28 # ... shell_command, file_reader, current_datetime
29 ]

```

#### Artifact 4 — The Agent Loop (Core Logic)

```

1 class AgentLoop:
2     def run(self, task: str) -> str:
3         messages = [{"role": "user", "content": task}]
4
5         for iteration in range(1, self.max_iterations + 1):
6             response = self.client.messages.create(
7                 model=self.model,
8                 max_tokens=2048,
9                 system=self.system_prompt,
10                tools=TOOLS,
11                messages=messages,
12            )
13
14            tool_results = []
15            final_text = ""
16
17            for block in response.content:
18                if block.type == "text":
19                    final_text += block.text
20                elif block.type == "tool_use":
21                    # Execute the tool locally
22                    result = execute_tool(
23                        block.name, block.input
24                    )
25                    tool_results.append({
26                        "type": "tool_result",
27                        "tool_use_id": block.id,
28                        "content": result,
29                    })

```

```

30
31     # If tools were called, feed results back
32     if tool_results:
33         messages.append({
34             "role": "assistant",
35             "content": response.content
36         })
37         messages.append({
38             "role": "user",
39             "content": tool_results
40         })
41         continue # Loop again
42
43     # No tools called = final answer
44     if response.stop_reason == "end_turn":
45         return final_text

```

### Pro Tip

Each tool function includes safety sandboxing: path traversal prevention for file operations, dangerous command blocking for shell execution, and a 10-second timeout. These are the same principles OpenClaw uses, scaled down for educational clarity.

## 5 Cost Awareness

### Token Consumption in Agentic Systems

OpenClaw consumes significantly more tokens than regular chat because each task triggers 5–10 API calls, and every call re-sends the full conversation context. A long session can burn 200K tokens just from carrying stale context.

| Model             | Input (\$/M) | Output (\$/M) | Est. Daily Cost  |
|-------------------|--------------|---------------|------------------|
| Claude Sonnet 4.6 | \$3          | \$15          | \$2–5 (light)    |
| Claude Opus 4.6   | \$5          | \$25          | \$5–15           |
| GPT-4o            | \$2.50       | \$10          | \$2–4            |
| Ollama (Local)    | Free         | Free          | Electricity only |

### Pro Tip

**Start new sessions regularly.** Context accumulation is the biggest cost driver — stale context gets re-sent on every call. All code artifacts in this tutorial include token tracking so you can monitor costs in real time.

## 6 Security: Eyes Wide Open

No responsible treatment of OpenClaw can omit the security discussion. As Malwarebytes observed: “Treating OpenClaw as a hardened productivity tool is wishful thinking — it be-

haves more like an over-eager intern with an adventurous nature, a long memory, and no real understanding of what should stay private.”

## 6.1 Known Risk Categories

### Security Risks — Summary

1. **Prompt Injection:** Malicious instructions embedded in data (emails, web pages) can hijack the agent. Cisco’s research team found third-party ClawHub skills performing data exfiltration without user awareness.
2. **The ClawHavoc Incident:** In February 2026, 12% of skills on ClawHub were found to contain malicious payloads designed to steal credentials.
3. **Excessive Agency:** Without strict permission boundaries, an agent meant for data lookup could accidentally delete records or exfiltrate data.
4. **Autonomous Overreach:** A CS student’s OpenClaw agent autonomously created a dating profile on MoltMatch and began screening matches without his direction.

## 6.2 Mitigations

- Run OpenClaw on a **dedicated machine** or **sandboxed environment** (Docker, VM)
- **Never install unreviewed skills** from ClawHub
- Use Nvidia’s **NemoClaw with OpenShell** for enterprise deployments (adds sandboxing, network guardrails, and privacy routing)
- Implement the **principle of least privilege** — give each agent only the permissions it needs
- Fix the **Canvas Host binding** to loopback immediately after installation (see Section 3.4)

## 7 The Bigger Picture: Why OpenClaw Matters

OpenClaw represents a genuine inflection point in AI — the shift from *generative models* (where competition was about model quality) to *agent ecosystems* (where competition moves to autonomy, orchestration, and tool integration).

### The Paradigm Shift

**2024–2025 Era:** Competition centred on model capability — who had the best reasoning, the largest context window, the most accurate outputs. OpenAI, Anthropic, and Google fought over benchmark scores.

**2026 Era:** Competition shifts to **agent orchestration**. An agent framework can be powerful even running on commodity models if the orchestration, memory, and tool ecosystem are superior. The model becomes a component; the agent is the product.

For those of us building production AI systems, the message is clear: the agent tooling layer is now the site of competition, not the model layer. If you are building on top of AI, you are building on top of an agent framework.

## 8 Getting Started Checklist

---

- ✓ Install Node.js v24+ and OpenClaw via npm
- ✓ Complete the onboarding wizard with your Anthropic API key
- ✓ **Immediately** fix the Canvas Host binding to loopback
- ✓ Clone the companion repository: `git clone https://github.com/anishiisc/openclaw`
- ✓ Run Artifact 1: Your First Agent (Section 4.1)
- ✓ Build a custom skill with Artifact 2 (Section 4.2)
- ✓ Experiment with multi-agent pipelines via Artifact 3 (Section 4.3)
- ✓ Study the agent loop from scratch with Artifact 4 (Section 4.4)
- ✓ Join the OpenClaw Discord for community support

## 9 References

---

- [1] OpenClaw Official Documentation. <https://docs.openclaw.ai/start/getting-started> (2026).
- [2] OpenClaw GitHub Repository. <https://github.com/openclaw/openclaw>. 347,000+ stars as of April 2026.
- [3] OpenClaw Python SDK (PyPI). <https://pypi.org/project/openclaw-sdk/>. Version 2.1.0.
- [4] shareAI-lab, “claw0: 0–1 Learn OpenClaw.” <https://github.com/shareAI-lab/claw0>. 10-session teaching repository with English, Chinese, and Japanese support.
- [5] K. Mehreen, “OpenClaw Explained: The Free AI Agent Tool Going Viral Already in 2026.” *KDnuggets*, April 2026. <https://www.kdnuggets.com/openclaw-explained-the-free-ai-agent-tool-g>
- [6] J. Prickett Morgan, “Nvidia Says OpenClaw Is To Agentic AI What GPT Was To Chatbots.” *The Next Platform*, March 2026. <https://www.nextplatform.com/ai/2026/03/17/nvidia-says-openclaw-is-to-agentic-ai-what-gpt-was-to-chattybots/>.
- [7] OpenClaw Python Integration Guide. <https://openclaw-ai.net/en/blog/openclaw-python-integration> (March 2026).
- [8] DigitalOcean, “How to Run OpenClaw.” <https://www.digitalocean.com/community/tutorials/how-to-run-openclaw> (January 2026).
- [9] ShareUHack, “OpenClaw Setup Tutorial 2026.” <https://www.shareuhack.com/en/posts/openclaw-setup-tutorial-2026> (April 2026).
- [10] Transparency Coalition for AI, “TCAI Guide: The Risks of AI Agents Built with OpenClaw.” <https://www.transparencycoalition.ai/news/tcai-guide-the-risks-of-ai-agents-built-with-o> (April 2026).
- [11] *Institutional Investor*, “OpenClaw: The AI Agent Institutional Investors Need to Understand — But Shouldn’t Touch.” February 2026. <https://www.institutionalinvestor.com/article/openclaw-ai-agent-institutional-investors-need-understand-shouldnt-touch>.

- [12] *TechTarget*, “OpenClaw and Moltbook Explained: The Latest AI Agent Craze.” April 2026.  
<https://www.techtarget.com/searchcio/feature/OpenClaw-and-Moltbook-explained-The-latest->
- [13] Wikipedia, “OpenClaw.” <https://en.wikipedia.org/wiki/OpenClaw>. Accessed April 2026.
- [14] Companion Code Repository (this tutorial). <https://github.com/anishiisc/openclaw>.

**Dr. Anish Roychowdhury**

Gen AI Architect · AI Educator · Practitioner

*Published Poet — “Gin and Ginger” (Kolkata International Book Fair 2026)*

✉ [anish.roychowdhury@gmail.com](mailto:anish.roychowdhury@gmail.com)

🐙 <https://github.com/anishiisc/openclaw>